



Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	• Armas Ordoñez Miguel Alejandro • Buri Camacho, María Soledad • Chamba Santín, Richard Ariel • Flores Gallardo, Emilio Josue • Rosillo Gaona, Odalis Camila
Asignatura	Sistemas Digitales
Ciclo	5 A
Unidad	1
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001
Tipo	Grupal
Título de la Práctica	Uso de Wokwi y Microchip Studio en microcontroladores AVR: arquitectura, puertos de E/S y operaciones a nivel de bit
Nombre del Docente	Ing. Luis Darío Sinche C.
Fecha	02 de octubre de 2026
Horario	Jueves 12h30 a 13h30 Viernes 11h30 a 13h30
Lugar	Laboratorio de computación (práctica 100 % simulada)
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la práctica

Objetivo general. Reconocer la arquitectura del microcontrolador ATmega328P y controlar sus periféricos de entradas/salidas digitales escribiendo y leyendo directamente sus registros mediante operaciones a nivel de bit, usando simulación en Wokwi y Microchip Studio.

Objetivos específicos:

- Relacionar los pines de la placa Arduino UNO con los puertos y bits del ATmega328P (PB, PC, PD).
- Configurar entradas y salidas digitales con los registros DDRx, PORTx y PINx usando máscaras (AND, OR, XOR, NOT y desplazamientos).
- Comparar el acceso mediante la API de Arduino con el acceso directo a registros, en tiempo de ejecución.
- Identificar en el código ensamblador generado las instrucciones lógicas y de manejo de bits del set de instrucciones AVR (SBI, CBI, SBIS, IN, OUT, ORI, EOR).

3. Materiales, reactivos, equipos y herramientas

No se requieren componentes físicos: toda la práctica es simulada. Componentes virtuales usados en Wokwi:

- 1 Arduino UNO (ATmega328P a 16 MHz).
- 4 LEDs (rojo, amarillo, verde y azul) y 4 resistencias de 220 Ω .
- 1 pulsador.

Computador con navegador web y conexión a internet.

- Wokwi (<https://wokwi.com>), con cuenta gratuita para guardar y compartir los proyectos.
- Microchip Studio 7 (solo Windows), con el simulador del ATmega328P.
- Si no dispone de Windows, use Compiler Explorer (<https://godbolt.org>) para la Parte E.
- Hoja de datos del ATmega328P y AVR Instruction Set Manual (Microchip Technology).
- Cuenta de GitHub y acceso al EVA para la entrega.

4. Procedimiento / Metodología ejecutada**5. Resultados****Parte A. Reconocimiento del entorno y del microcontrolador**

- A1. Ingrese a wokwi.com, cree un proyecto nuevo con Arduino Uno y guárdelo con el nombre APE1_<apellido1>_<apellido2>.
- A2. En la placa simulada, identifique el chip ATmega328P, el LED integrado “L” y el pin D13. Con la sección de Fundamento teórico, indique a qué puerto y bit corresponde D13.
- A3. Escriba el Código 1 (línea base con la API de Arduino), inicie la simulación y verifique que el LED “L” parpadea. Código 1. Blink con la API de Arduino

Código 1. Blink con la API de Arduino

```
1 // APE 1 - Código 1: Blink con la API de Arduino ( línea base)
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulación en Wokwi
3 void setup ( ) {
4   pinMode ( 13 , SALIDA ) ; // D13 = PB5 (LED integrado "L")
5 }
6 void loop ( ) {
7   digitalWrite ( 13 , ALTO ) ;
8   retraso ( 500 ) ;
9   digitalWrite ( 13 , BAJO ) ;
10  retraso ( 500 ) ;
```

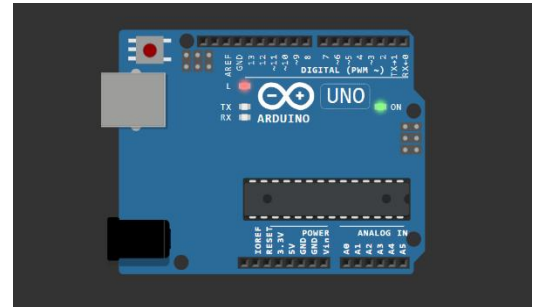


Parte B. El mismo comportamiento, ahora con registros

- B1. Reemplace el código por el Código 2 y verifique que el comportamiento es
- idéntico al del Código 1.

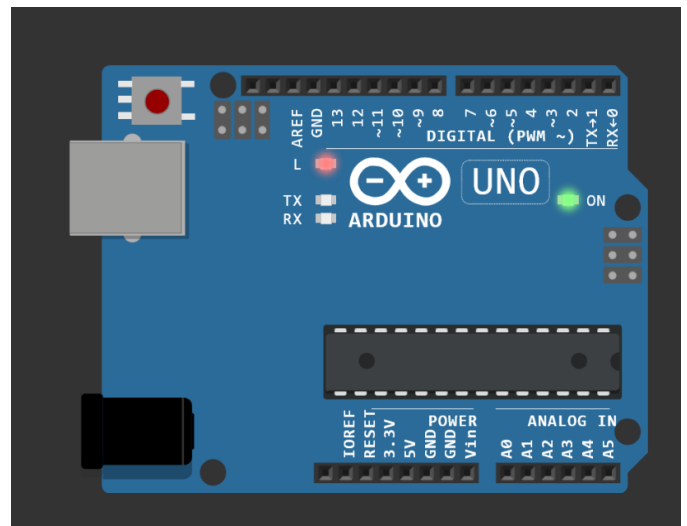
Código 2. Blink escribiendo registros

```
1 // APE 1 - Código 2: el mismo Blink, ahora escribiendo registros
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulación en Wokwi
3 // Regla: no se usa pinMode() ni digitalWrite()
4 void setup() {
5   DDRB |= (1 << DDB5); // PB5 (D13) como salida; los demás bits no cambian
6 }
7 void loop() {
8   PORTB ^= (1 << PORTB5); // XOR: conmuta solo el bit 5
9   retraso(500); // 500 ms encendido / 500 ms apagado
10 }
```



- B2. Sustituya $PORTB \wedge= (1 \ll PORTB5)$; por un par de líneas que enciendan ($|=$) y apaguen ($\&= \sim$) el LED con un delay (500) entre ellas. Verifique.

```
27
28 // APE 1 - Parte B2
29 // Encendido y apagado usando OR y AND
30
31 void setup() {
32   // PB5 (D13) como salida
33   DDRB |= (1 << DDB5);
34 }
35
36 void loop() {
37   // ENCENDER LED
38   // Coloca el bit 5 de PORTB en 1
39   PORTB |= (1 << PORTB5);
40
41   delay(500);
42
43   // APAGAR LED
44   // Coloca el bit 5 de PORTB en 0
45   PORTB &= ~(1 << PORTB5);
46
47   delay(500);
48 }
```



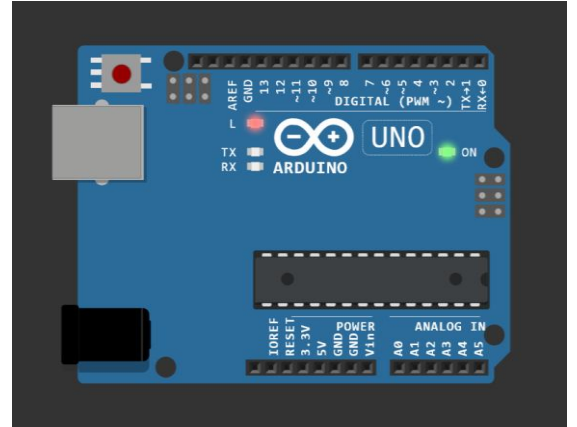
- B3. Sustituya la conmutación por $PINB = (1 \ll PINB5)$; y verifique. Anote qué

característica del ATmega328P lo permite.

```

36 void loop() {
47   delay(500);
48 }*/
49
50 // APE 1 - Parte B3
51 // Conmutación escribiendo directamente en PINB
52
53 void setup() {
54   // PB5 (D13) como salida
55   DDRB |= (1 << DDB5);
56 }
57
58 void loop() {
59   // Escribir un 1 en PINB5 conmuta el bit correspondiente de PORTB
60   PINB = (1 << PINB5);
61
62   delay(500);
63 }

```

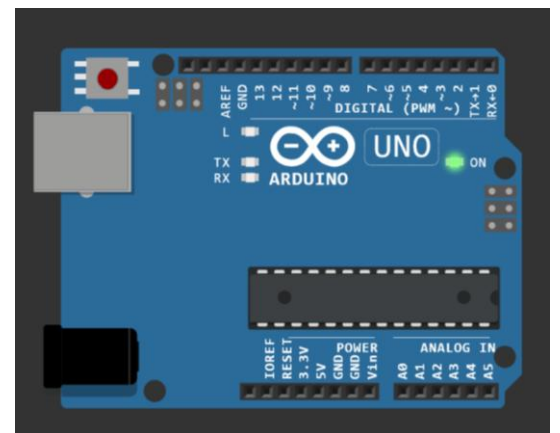


- B4. Pruebe $PORTB = (1 \ll PORTB5)$; (asignación en lugar de OR). Funciona aquí, pero explique en el informe qué les pasaría a los otros bits de PORTB si hubiera más salidas conectadas en ese puerto. Lo comprobará en la Parte C.

```

66 // Uso de asignación directa sobre PORTB
67
68 void setup() {
69   // PB5 (D13) como salida
70   DDRB |= (1 << DDB5);
71 }
72
73 void loop() {
74   // Coloca PB5 en 1,
75   // PERO también coloca todos los demás bits de PORTB en 0
76   PORTB = (1 << PORTB5);
77
78   delay(500);
79
80   // Apaga todo PORTB
81   PORTB = 0;
82
83   delay(500);
84 }

```



Parte C. Circuito y laboratorio de bits

- C1. Arme en Wokwi el circuito de la Tabla 3. Puede hacerlo a mano o pegar el contenido del Anexo A en la pestaña diagram.json.

Tabla 3. Conexiones pin a pin

Componente	Pin del componente	Pin Arduino UNO	Puerto / bit	Configuración
LED0 (rojo) + R0 220 Ω	Ánodo (A) vía R0	D8	PB0	Salida
LED1 (amarillo) + R1 220 Ω	Ánodo (A) vía R1	D9	PB1	Salida
LED2 (verde) + R2 220 Ω	Ánodo (A) vía R2	D10	PB2	Salida
LED3 (azul) + R3 220 Ω	Ánodo (A) vía R3	D11	PB3	Salida
LED0 ... LED3	Cátodo (C)	GND	—	—
Pulsador	1.l	D2	PD2	Entrada con pull-up interna
Pulsador	2.l	GND	—	—
LED integrado "L"	—	D13	PB5	Salida (latido)

Nota sobre pines: No use D0 ni D1, porque son la UART que usa el Monitor Serie. El pulsador va a GND porque se usa la resistencia pull-up interna, así que se lee 1 suelto y 0 presionado (activo en bajo). En el diagram.json, el rebote del pulsador está desactivado ("bounce": "0"); en hardware real habrá que tratar el rebote.

- C2. Cargue el Código 3, inicie la simulación y abra el Monitor Serie (9600 baudios).

```
void loop() {
  if (millis() - tAnterior >= PERIODO_MS) {
    switch (modo) {
      case 1:
        break;

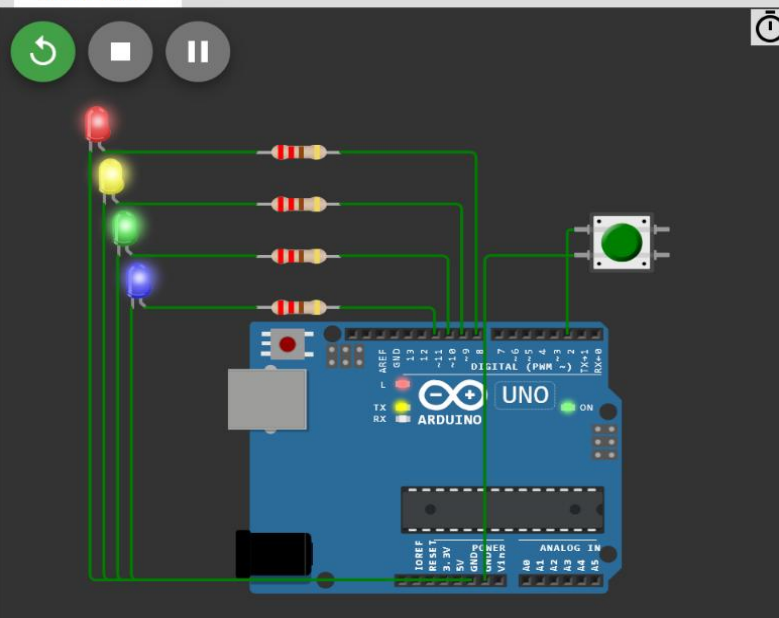
        // -----
        // MODO 2
        // Alternancia mediante XOR
        // -----
      case 2:
        patron ^= MASK_LEDS;
        break;
    }

    // Mostrar patrón en los LEDs
    escribirLeds(patron);

    // Conmutar LED integrado D13 / PB5
    PINB = (1 << PINB5);

    // Mostrar PORTB en monitor serie
    imprimirBin8("PORTB = ", PORTB);
  }
}
```

Simulation



```
PORTB = 00101001
PORTB = 00001010
PORTB = 00101011
PORTB = 00001100
PORTB = 00101101
PORTB = 00001110
PORTB = 00101111
```

- C3 Antes de mirar el Monitor Serie, calcule a mano los resultados de `demoOperaciones()` y luego compárelos con la salida.

Operación	Resultado
a	11001100
b	10101010
a & b	10001000
a b	11101110
a ^ b	01100110
(uint8_t)~a	00110011
a << 1	10011000
a >> 2	00110011

- C4. Presione el pulsador y
describa los tres modos. Para cada modo, explique qué operación a nivel de bit produce el patrón.

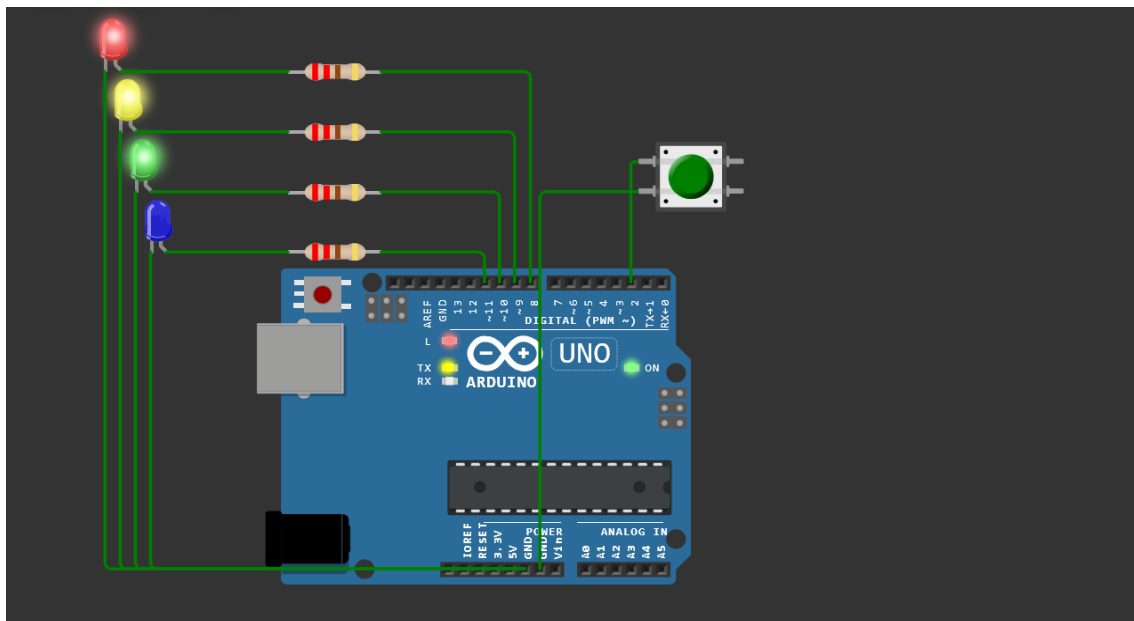
Existen 3 modos:

En el modo 0: los cuatro LEDs representan un contador binario de 4 bits. El valor de patrón se incrementa en una unidad cada 300 ms. La operación AND con `MASK_LEDS` (00001111) limita el resultado a los cuatro bits correspondientes a `PB0–PB3`.

En el modo 1: se realiza una rotación circular hacia la izquierda. Patrón << 1: Desplaza el bit encendido hacia la izquierda y patrón >> 3: Recupera el bit que sale del extremo para introducirlo nuevamente en la posición menos significativa. Por ello, un LED encendido se desplaza de forma circular entre los cuatro LEDs.

En el modo 2: se utiliza XOR con la máscara 1111. Como XOR con 1 invierte el bit, los cuatro bits cambian de estado cada 300 ms. De esta manera, el patrón alterna entre 0101 y 1010.

- C5. Observe la línea `PORTB = ...` del Monitor Serie e identifique qué bits corresponden a los LEDs y cuál al LED “L”.



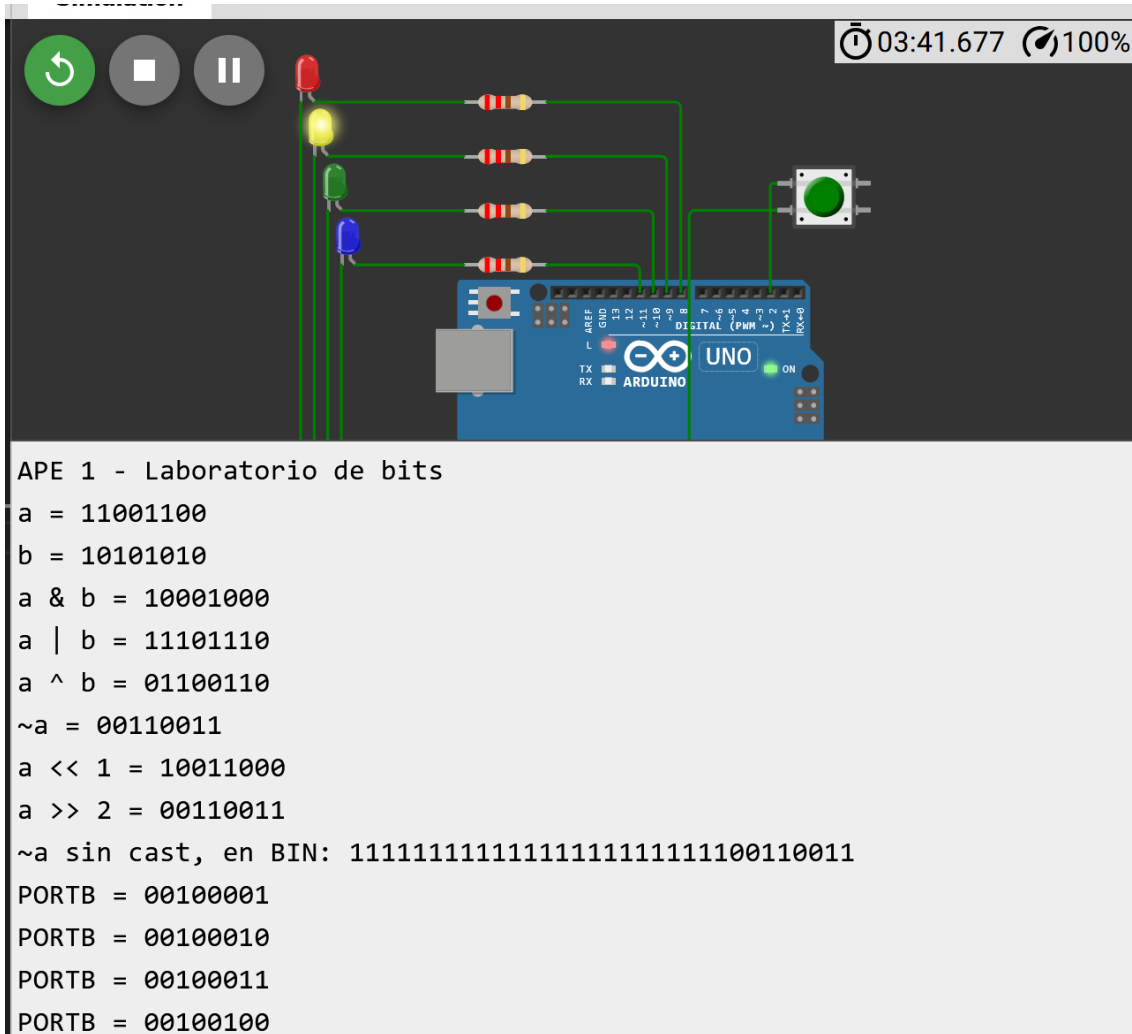
```
PORTB = 00101011
PORTB = 00001100
PORTB = 00101101
PORTB = 00001110
PORTB = 00101111
PORTB = 00000000
PORTB = 00100001
```

Los bits PB0–PB3 de PORTB corresponden a los cuatro LEDs externos. El bit PB5 corresponde al LED integrado L. Por lo tanto, los valores mostrados por el Monitor Serie permiten observar simultáneamente el patrón de los LEDs externos y el estado del LED integrado.

- C6. Cambie escribirLeds() por PORTB = valor; y describa qué ocurre con el LED “L”. Relacione el resultado con el paso B4 y restaure la versión original.

Con el código:

```
void escribirLeds(uint8_t valor) {
    PORTB = valor;
}
```



```
h.ino      Diagram.json      Library Manager
void imprimirBin8(const char *etiqueta, uint8_t v) {
    for (int8_t i = 7; i >= 0; i--) {
        Serial.print((v >> i) & 1);
    }

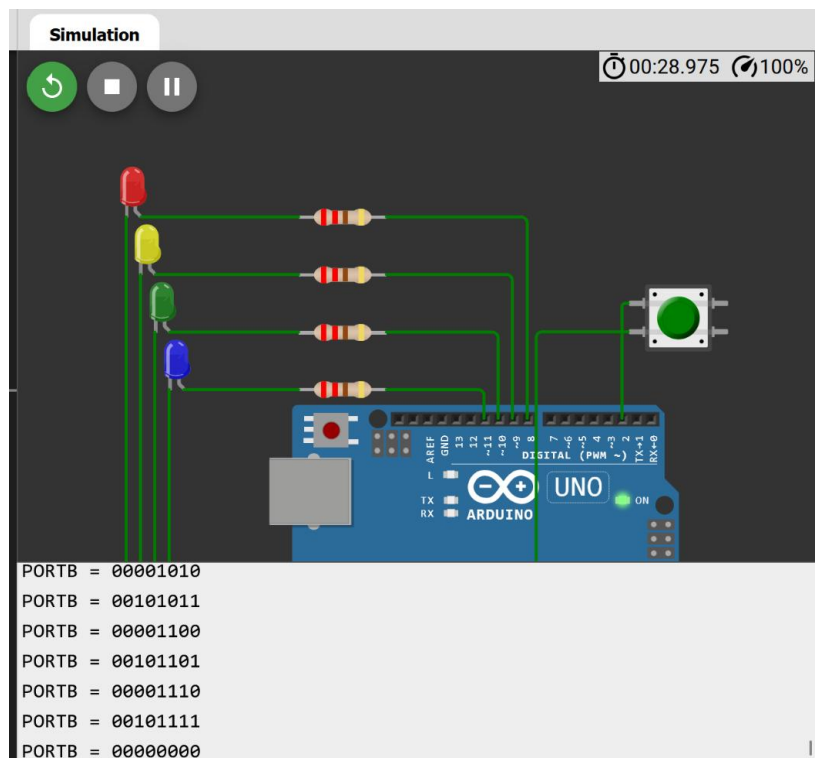
    Serial.println();
}

// Escribe solamente en PB0..PB3
void escribirLeds(uint8_t valor) {
    PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
}
//void escribirLeds(uint8_t valor){
//    PORTB = valor;
//}
*/
// Demostración de operaciones binarias
void demoOperaciones() {

    uint8_t a = 0b11001100;
    uint8_t b = 0b10101010;

    imprimirBin8("a = ", a);
    imprimirBin8("b = ", b);

    imprimirBin8("a & b = ", a & b);
    imprimirBin8("a | b = ", a | b);
    imprimirBin8("a ^ b = ", a ^ b);
}
```



Al sustituir la operación de lectura-modificación-escritura por `PORTB = valor`, el LED integrado `L` deja de mantener correctamente su conmutación. Esto sucede porque la asignación directa reemplaza todos los bits de `PORTB`, incluido `PB5`. La implementación original utiliza una máscara para modificar únicamente `PB0–PB3` y conservar el estado de los demás bits.

- C7. Complete la Tabla 4 en el informe.

Tabla 4. Configuración de registros

Pin	Función	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED0 encendido	1	1	—
D11 (PB3)	LED3 apagado	1	0	—
D2 (PD2)	Pulsador suelto	0	1	1
D2 (PD2)	Pulsador presionado	0	1	0
D13 (PB5)	LED L encendido	1	1	—

Parte D. Costo de la API frente al acceso a registros

- D1. En un proyecto nuevo de Wokwi (Arduino Uno, sin componentes adicionales), cargue el Código 4 y ejecútelo tres veces.

```

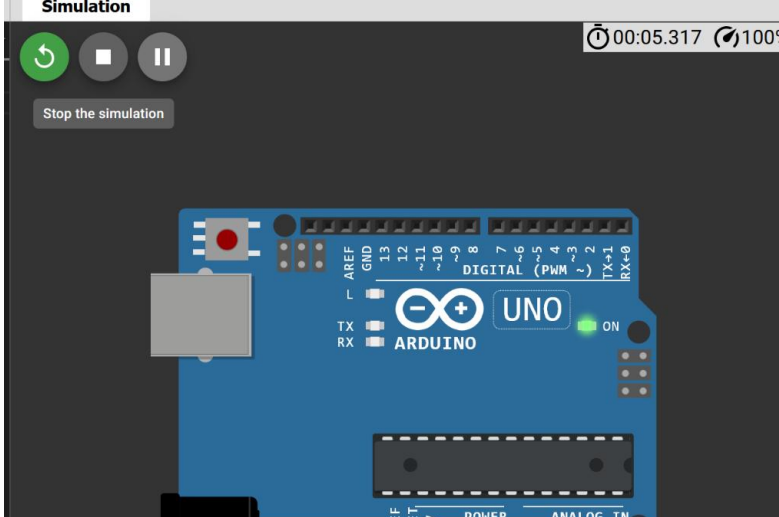
1.ino  diagram.json  Library Manager
// APE 1 - Código 4: costo de digitalWrite() frente al
// Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulación
const uint16_t N = 10000;
void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);
  unsigned long t0 = micros();
  for (uint16_t i = 0; i < N; i++) {
    digitalWrite(13, HIGH);
    digitalWrite(13, LOW);
  }
  unsigned long t1 = micros();
  for (uint16_t i = 0; i < N; i++) {
    PORTB |= (1 << PORTB5);
    PORTB &= ~(1 << PORTB5);
  }
  unsigned long t2 = micros();
  Serial.print(F("digitalWrite : "));
  Serial.print((t1 - t0) / (2.0 * N), 3);
  Serial.println(F(" us por escritura"));
  Serial.print(F("Registro : "));
  Serial.print((t2 - t1) / (2.0 * N), 3);
  Serial.println(F(" us por escritura"));
}
void loop() {
}

```

Simulation

00:05.317 100%

Stop the simulation



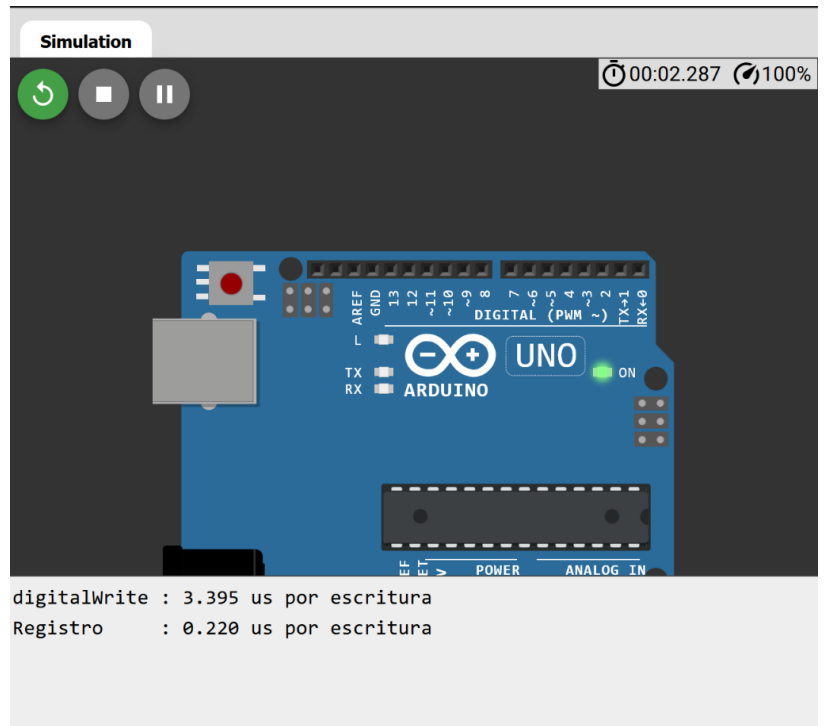
digitalWrite : 3.395 us por escritura

Registro : 0.220 us por escritura

```

.ino  diagram.json  Library Manager
// APE 1 - Código 4: costo de digitalWrite() frente a
// Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulac
const uint16_t N = 10000;
void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);
  unsigned long t0 = micros();
  for (uint16_t i = 0; i < N; i++) {
    digitalWrite(13, HIGH);
    digitalWrite(13, LOW);
  }
  unsigned long t1 = micros();
  for (uint16_t i = 0; i < N; i++) {
    PORTB |= (1 << PORTB5);
    PORTB &= ~(1 << PORTB5);
  }
  unsigned long t2 = micros();
  Serial.print(F("digitalWrite : "));
  Serial.print((t1 - t0) / (2.0 * N), 3);
  Serial.println(F(" us por escritura"));
  Serial.print(F("Registro : "));
  Serial.print((t2 - t1) / (2.0 * N), 3);
  Serial.println(F(" us por escritura"));
}
void loop() {
}

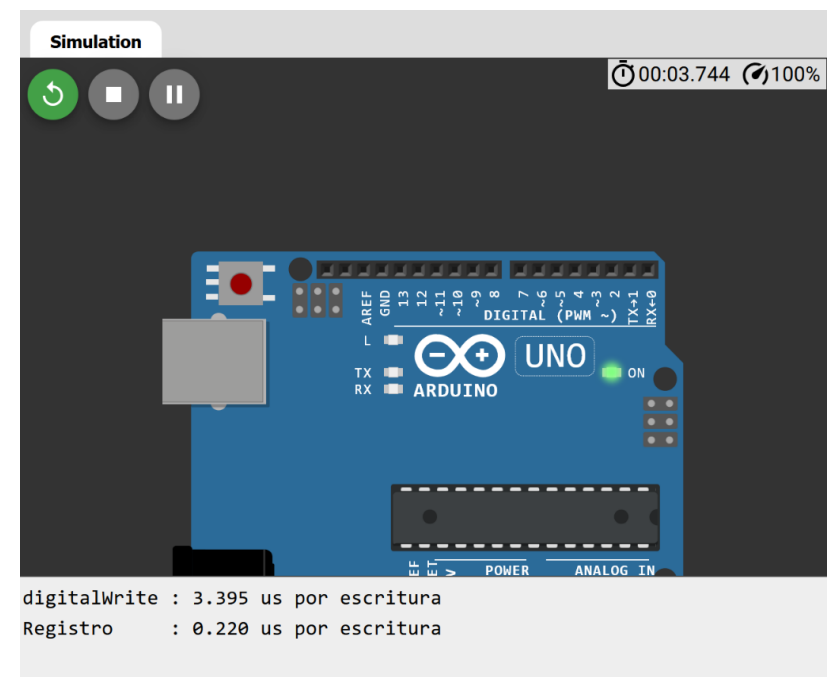
```



```

sketch.ino  diagram.json  Library Manager
1 // APE 1 - Código 4: costo de digitalWrite() frente
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simula
3 const uint16_t N = 10000;
4 void setup() {
5   Serial.begin(9600);
6   pinMode(13, OUTPUT);
7   unsigned long t0 = micros();
8   for (uint16_t i = 0; i < N; i++) {
9     digitalWrite(13, HIGH);
10    digitalWrite(13, LOW);
11  }
12  unsigned long t1 = micros();
13  for (uint16_t i = 0; i < N; i++) {
14    PORTB |= (1 << PORTB5);
15    PORTB &= ~(1 << PORTB5);
16  }
17  unsigned long t2 = micros();
18  Serial.print(F("digitalWrite : "));
19  Serial.print((t1 - t0) / (2.0 * N), 3);
20  Serial.println(F(" us por escritura"));
21  Serial.print(F("Registro : "));
22  Serial.print((t2 - t1) / (2.0 * N), 3);
23  Serial.println(F(" us por escritura"));
24  }
25  void loop() {
26  }
27

```

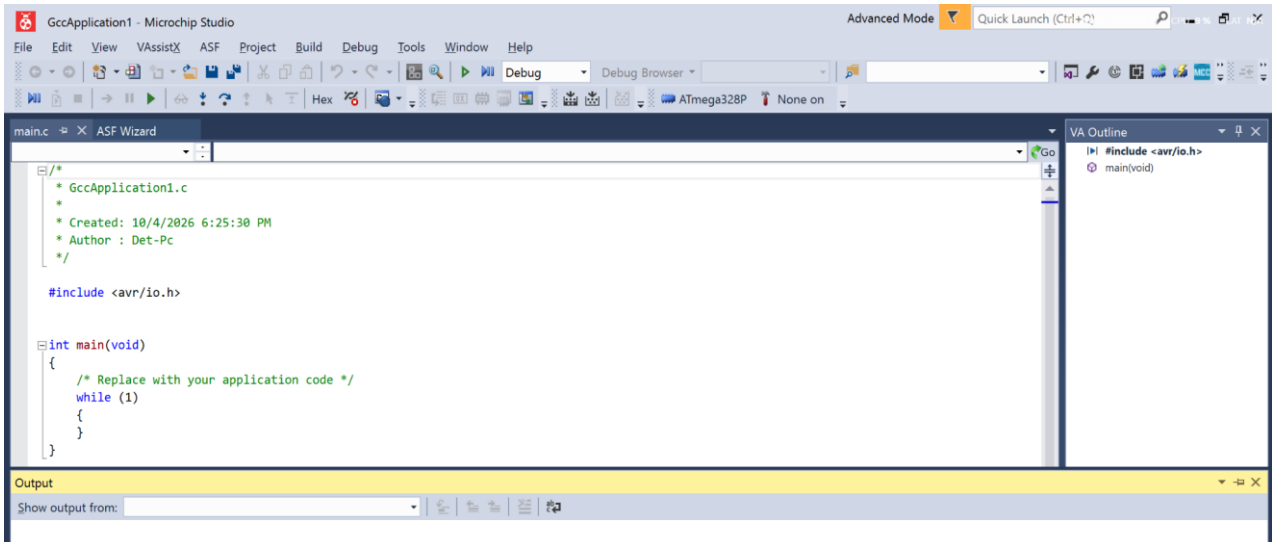


Código 4. Medición de tiempo de escritura

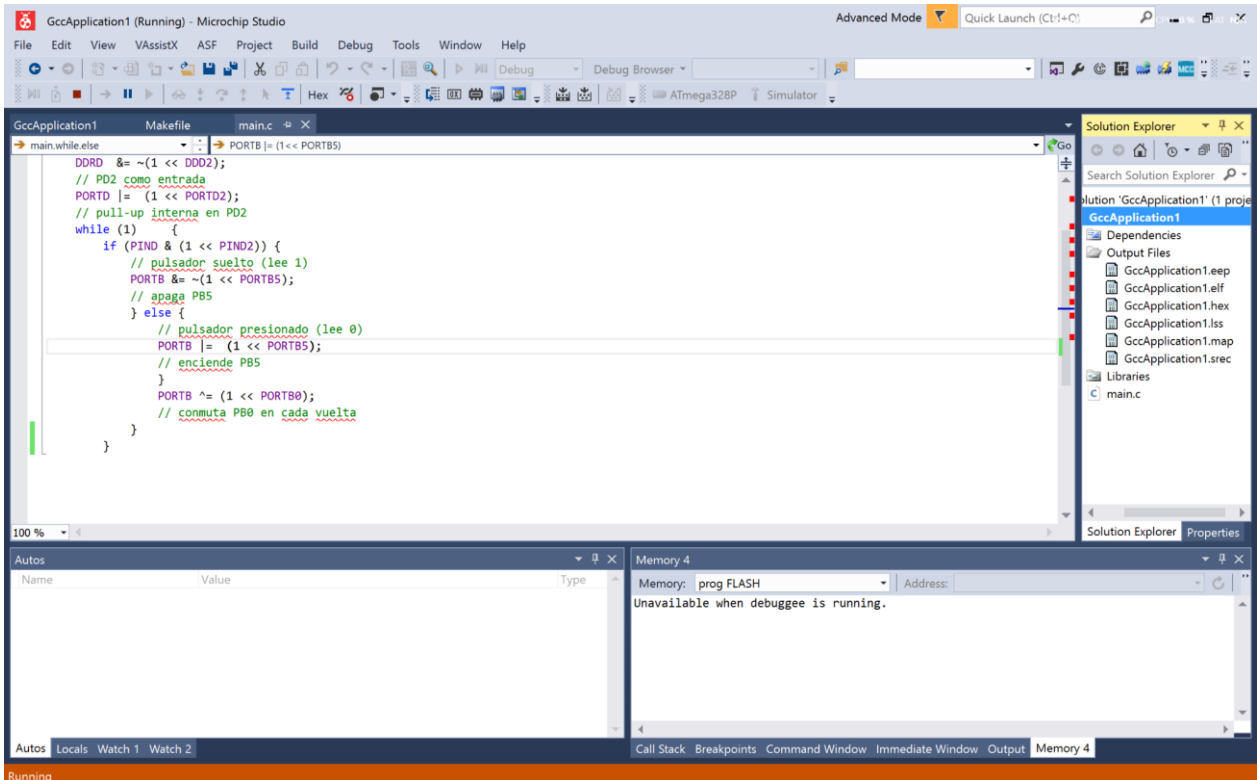
- D2. Registre los tiempos de las tres ejecuciones, calcule el promedio y la relación digitalWrite/registro. Los tiempos incluyen la sobrecarga del bucle y de la interrupción del Timer0, por lo que interesa la comparación, no el valor absoluto

Parte E. Del C al set de instrucciones con Microchip Studio

- E1. En Microchip Studio: File › New › Project › GCC C Executable Project, dispositivo ATmega328P.



- E2. Reemplace el contenido de main.c por el Código 5 y compile (Build › Build Solution, F7). Verifique que no haya errores.



Código 5. C puro para Microchip Studio (sin Arduino)

- E3. Abra el archivo .lss (Solution Explorer > Output Files). Localice la función main y complete la Tabla 5 con las instrucciones generadas para cada línea de C y sus ciclos según el AVR Instruction Set Manu

Tabla 5. Correspondencia C – ensamblador (para completar)

Línea en C	Instrucción(es) AVR generada(s)	Ciclos
DDRB = (1 << DDB5) (1 << DDB0);	IN r24, 0x04 → ORI r24, 0x21 → OUT 0x04, r24	3
DDRD &= ~(1 << DDD2);	IN r24, 0x0A → ANDI r24, 0xFB → OUT 0x0A, r24	3
PORTD = (1 << PORTD2);	IN r24, 0x0B → ORI r24, 0x04 → OUT 0x0B, r24	3
if (PIND & (1 << PIND2))	SBIS 0x09, 2 → RJMP .+8	2 si PD2=1 / 3 si PD2=0
PORTB &= ~(1 << PORTB5);	IN r24, 0x05 → ANDI r24, 0xDF → OUT 0x05, r24 → RJMP .+6	5
PORTB = (1 << PORTB5);	IN r24, 0x05 → ORI r24, 0x20 → OUT 0x05, r24	3
PORTB ^= (1 << PORTB0);	IN r25, 0x05 → LDI r24, 0x01 → EOR r24, r25 → OUT 0x05, r24	4

- E4. Seleccione el simulador (Project > Properties > Tool > Simulator) e inicie la depuración con Debug > Start Debugging and Break (Alt+F5).
- E5. Abra Debug > Windows > I/O, seleccione PORTB y PORTD y avance instrucción por instrucción (F10/F11). Observe cómo cambian DDRB, PORTB y PORTD. Simule el pulsador modificando el bit 2 de PIND en la vista I/O y observe el efecto en PB5. Si el simulador no refleja la pull-up, ponga PIND2 en 1 a mano para representar “suelto”.



Two screenshots of the Microchip Studio IDE showing the debugging process for a GccApplication1 project on an ATmega328P microcontroller.

Top Screenshot: The I/O View shows the I/O Port (PORTB) selected. The I/O View table lists the following registers:

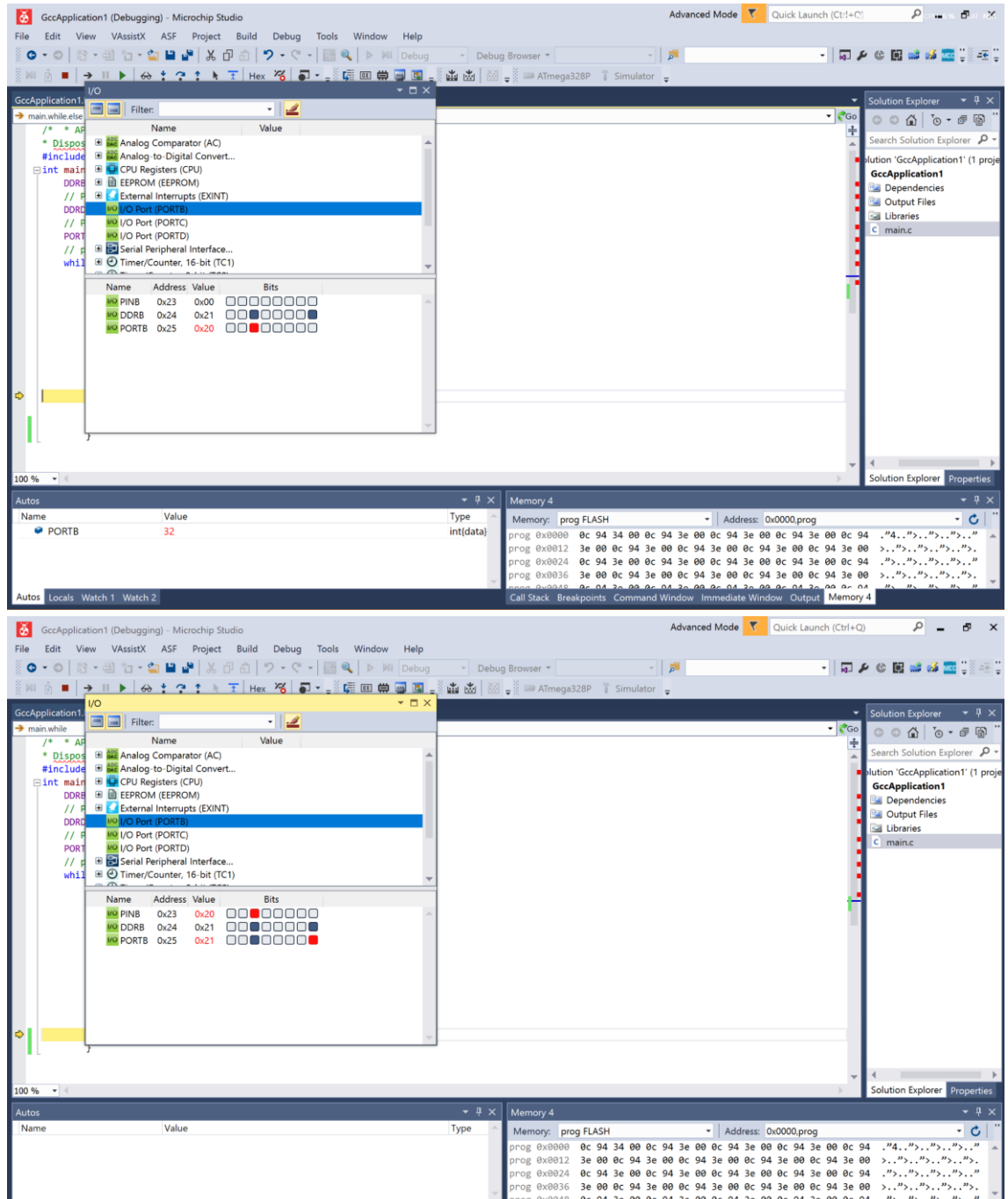
Name	Address	Value	Bits
PINB	0x23	0x00	00000000
DDRB	0x24	0x21	00000001
PORTB	0x25	0x00	00000000

The Autos window shows the variable `DDRD` with a value of 0. The Memory window shows the program memory (prog FLASH) starting at address 0x0000.

Bottom Screenshot: The I/O View shows the I/O Port (PORTD) selected. The I/O View table lists the following registers:

Name	Address	Value	Bits
PIND	0x29	0x00	00000000
DDRD	0x2A	0x00	00000000
POR...	0x2B	0x04	00000001

The Autos window shows the variable `PIND` with a value of 0. The Memory window shows the program memory (prog FLASH) starting at address 0x0000.



- E6. En Debug > Windows > Processor Status, use el contador de ciclos para medir cuántos ciclos tarda una vuelta del while(1) con el pulsador suelto y con el pulsador presionado.

Pulsador - Suelto



Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

GccApplication1 (Debugging) - Microchip Studio

File Edit View VAssistX ASF Project Build Debug Tools Window Help

Debug Browser - Debug

GccApplication1.lss GccApplication1 Makefile main.c

```
/* APE 1 -Codigo 5: Microchip Studio (C)
 * Dispositivo: ATmega328P | Toolchain: AVR/
 * #include <avr/io.h>
 */
int main(void) {
    DDRC |= (1 << DDRC5) | (1 << DDRC0);
    // PB5 y PB0 como salidas
    DDRC &= ~(1 << DDRC2);
    // PD2 como entrada
    PORTD |= (1 << PORTD2);
    // pull-up interna en PD2
    while (1) {
        if (PIND & (1 << PIND2)) {
            // pulsador suelto (lee 1)
            PORTB &= ~(1 << PORTB5);
            // apaga PB5
        } else {
            // pulsador presionado (lee 0)
            PORTB |= (1 << PORTB5);
            // enciende PB5
            PORTB ^= (1 << PORTB0);
            // conmuta PB0 en cada vuelta
        }
    }
}
```

Processor Status

Name	Value
Program Counter	0x0000004B
Stack Pointer	0x08FD
X Register	0x0000
Y Register	0x08FF
Z Register	0x0000
Status Register	0x00000020
Cycle Counter	35
Frequency	1.000 MHz
Stop Watch	35.00 µs

Registers

Register	Value
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00

Autos

Name	Value	Type
PORTB	33	int(data)

Memory 4

Memory: prog FLASH Address: 0x0000,prog

Call Stack Breakpoints Command Window Immediate Window Output Memory 4

Stopped Ln 14 Col 1 Ch 1 INS

Pulsador - Presionado

GccApplication1 (Debugging) - Microchip Studio

File Edit View VAssistX ASF Project Build Debug Tools Window Help

Debug Browser - Debug

GccApplication1.lss GccApplication1 Makefile main.c

```
/* APE 1 -Codigo 5: Microchip Studio (C)
 * Dispositivo: ATmega328P | Toolchain: AVR/
 * #include <avr/io.h>
 */
int main(void) {
    DDRC |= (1 << DDRC5) | (1 << DDRC0);
    // PB5 y PB0 como salidas
    DDRC &= ~(1 << DDRC2);
    // PD2 como entrada
    PORTD |= (1 << PORTD2);
    // pull-up interna en PD2
    while (1) {
        if (PIND & (1 << PIND2)) {
            // pulsador suelto (lee 1)
            PORTB &= ~(1 << PORTB5);
            // apaga PB5
        } else {
            // pulsador presionado (lee 0)
            PORTB |= (1 << PORTB5);
            // enciende PB5
            PORTB ^= (1 << PORTB0);
            // conmuta PB0 en cada vuelta
        }
    }
}
```

Processor Status

Name	Value
Program Counter	0x0000004F
Stack Pointer	0x08FD
X Register	0x0000
Y Register	0x08FF
Z Register	0x0000
Status Register	0x00000020
Cycle Counter	24
Frequency	1.000 MHz
Stop Watch	24.00 µs

Registers

Register	Value
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00

Autos

Name	Value	Type
PORTB	0	int(data)

Memory 4

Memory: prog FLASH Address: 0x0000,prog

Call Stack Breakpoints Command Window Immediate Window Output Memory 4

6. Preguntas de control

1. ¿Qué diferencia a un microcontrolador de un microprocesador? Enumere los Bloques del ATmega328P que identificó en la práctica.

- Un microcontrolador integra en un solo chip la CPU, la memoria de programa y los periféricos, mientras que un microprocesador necesita memoria y periféricos externos. Los bloques del ATmega328P identificados a través de las actividades de la práctica son: los puertos de entradas y salidas digitales (E/S) controlados por registros, las memorias Flash, SRAM y EEPROM, la CPU (analizada mediante la ejecución de instrucciones AVR y ciclos de reloj), y el temporizador Timer0 (identificado por la mención a su interrupción al realizar mediciones de tiempo).

2. ¿Qué implica que el ATmega328P tenga arquitectura Harvard? Relacione su respuesta con el uso de la macro F(...) en el Código 3.

- La arquitectura Harvard implica que el microcontrolador utiliza buses separados para la memoria de programa y la memoria de datos. La relación con la macro F es que, al usar el pulsado, cada uno trabaja de una manera independiente, por lo que es como si cada LED tuviera un bus diferente trabajando de manera asíncrona.

3. Explique por qué $PORTB = (1 \ll PORTB5)$; puede alterar otras salidas y $PORTB |= (1 \ll PORTB5)$; no, usando lo observado en los pasos B4 y C6.

- La instrucción $PORTB = (1 \ll PORTB5)$; utiliza una asignación directa que reemplaza el valor de todo el registro de 8 bits; coloca un 1 en el bit 5 y ceros en todos los demás, lo que sobrescribe y altera cualquier otra salida activa en ese puerto. Por el contrario, la instrucción $PORTB |= (1 \ll PORTB5)$; aplica un operador lógico OR con una máscara que solo tiene el bit 5 en 1, lo que permite modificar únicamente ese bit sin alterar el estado de los demás pines del puerto.

4. Para usar PD2 como entrada con pull-up, ¿qué valores deben tener DDRD2 y PORTD2? ¿Qué se lee en PIND2 con el pulsador suelto y presionado, y por qué?

- Para que el pin actúe como entrada, el bit correspondiente en el registro DDR (DDRD2) debe estar en 0. Para activar la resistencia pull-up interna en un pin de entrada, el bit en el registro PORT (PORTD2) debe estar en 1. Con el pulsador suelto se lee un 1 en PIND2 porque la resistencia pull-up interna mantiene el nivel del pin en VCC. Con el pulsador presionado se lee un 0 porque el circuito se cierra hacia GND, funcionando de forma activa en bajo.

5. Calcule a mano $0b11001100 \& 0b10101010$, $|$ y $\wedge$, y verifique sus resultados con el Monitor Serie.



0b = Binario

0x = Hexadecimal

Operaciones binarias

$a = 0b11001100$

$b = 0b10101010$

Operación AND bit a bit (&)

a: 11001100

b: 10101010

a&b: 10001000 (0x88)

Operación OR bit a bit (|)

a: 11001100

b: 10101010

a|b: 11101110 (0xEE)

Operación XOR bit a bit (^)

a: 11001100

b: 10101010

a^b: 01100110 (0x66)

6. Si a es de tipo uint8_t, ¿por qué Serial.println(~a, BIN) imprime muchos más de 8 bits? ¿Cómo lo corrige el Código 3?

- En C o C++, cuando se aplica el operador NOT (~) a una variable entera de menor tamaño que int (como el uint8_t), ocurre la promoción a entero (integer promotion). La variable *a* de 8 bits se convierte automáticamente a un int con signo de 16 bits (o 32 bits según la plataforma).
- Al negar $a = 0b11001100$ (204 en decimal), en 16 bits pasa a ser $0b0000000011001100$. Su complemento bit a bit resulta en $0b1111111100110011$ (o un número negativo en complemento a dos). Por ello, Serial.println() imprime los 16 bits completos con los unos extendidos a la izquierda.
- ¿Y cómo corrige el Código 3? El Código 3 lo soluciona haciendo un cast (conversión explícita de tipo) a 8 bits: (uint8_t)~a. De esta forma, fuerza a truncar el resultado nuevamente a un entero sin signo de 8 bits antes de pasarlo a la función de impresión. Además, para la función custom imprimirBin8(), recorre manualmente del bit 7 al bit 0 enmascarando cada posición ($v \gg i$) & 1.

7. Con los datos de la Parte D, explique por qué digitalWrite() es más lento que el acceso directo a registros. ¿Cuándo conviene cada enfoque (portabilidad frente a rendimiento)?

Es más lento digitalWrite() porque es una función de alto nivel que llega a incluir múltiples comprobaciones de seguridad e intermediación en tiempo de ejecución, por decir:

1. Debe traducir el número de pin de Arduino (ej. pin 13) al puerto (PORTB) y bit exacto (PB5) correspondiente mediante tablas guardadas en memoria Flash (pgm_read_byte).
 2. Y utiliza secciones críticas (desactiva e interrumpe temporalmente las interrupciones globales con cli()/sei()) para asegurar un acceso atómico.
- Ahora, mientras que `PORTB |= ...` Se compila directamente en una o dos instrucciones de ensamblador de un solo ciclo de reloj (como SBI o CBI), ejecutándose en menor tiempo a 16MHz.
 - ¿Entonces cuándo se usa el digitalWrite() o el acceso directo?
 - El digitalWrite se lo utiliza por la “portabilidad y facilidad”, ya que conviene en prototipado rápido, proyectos educativos o código que deba ejecutarse sin cambios en diferentes arquitecturas de microcontroladores.
 - Ahora el acceso directo se lo utiliza por “Rendimiento” porque cuando se requiere un control de tiempo preciso (nanosegundos), espacio en memoria y ciclos de reloj en aplicaciones de tiempo real e industriales. Así que busca la

8. ¿Qué instrucción AVR generó `PORTB |= (1 << PORTB5);` y cuántos ciclos toma? ¿Por qué `PORTB ^= (1 << PORTB0);` no se tradujo a una sola instrucción?

- La expresión `PORTB |= (1 << PORTB5);` generó la instrucción SBI (*Set Bit in I/O Register*) y toma 2 ciclos de reloj; se ejecuta de forma directa porque la arquitectura AVR posee una instrucción atómica especial para poner en 1 un bit individual en registros de entrada/salida. En cambio, `PORTB ^= (1 << PORTB0);` no se tradujo a una sola instrucción porque el microcontrolador AVR no cuenta con una instrucción atómica directa para realizar la operación lógica XOR (^=) en un registro de E/S, obligando al compilador a realizar una secuencia de tres pasos de *Lectura - Modificación - Escritura*: IN (leer el estado actual de PORTB), EOR (aplicar el XOR lógico en un registro de trabajo) y OUT (escribir el resultado final de vuelta en PORTB).

9. Sostenibilidad: Indique dos formas en que lo trabajado (simular antes de comprar o montar, acceso eficiente a los periféricos) contribuye a reducir el consumo energético o los residuos electrónicos.

Dos formas en las cuales esta práctica contribuye a la sostenibilidad son:

1. Optimización del consumo energético y de diseño: La simulación previa permite evaluar la interacción de los componentes en diversos escenarios de operación (variaciones de voltaje, corriente y resistencia) sin necesidad de mantener un circuito físico encendido durante la fase de pruebas, eliminando el consumo eléctrico innecesario.
2. Reducción de residuos electrónicos (e-waste) y costos: Al validar el funcionamiento mediante simulación antes del montaje físico, se evita el desgaste, daño accidental o descarte de

componentes por errores de cálculo. Esto previene la generación de basura electrónica y optimiza el uso de recursos económicos al adquirir únicamente los componentes requeridos.

10. Ventajas y limitaciones de simular en Wokwi frente al hardware real (rebote del pulsador, límites de corriente, tiempos).

Simular en Wokwi agiliza el desarrollo de software en un entorno seguro, pero difiere del hardware real al idealizar los comportamientos físicos de los componentes.

Ventajas

- Pruebas seguras y rápidas: Permite validar la lógica del código en arquitecturas como ESP32, Arduino o Raspberry Pi Pico sin riesgo de dañar componentes ni lidiar con conexiones físicas defectuosas.
- Herramientas de depuración: Integra un analizador lógico virtual y puerto serie sin ruido externo para inspeccionar señales con precisión.
- Cero costos de material: Permite probar circuitos completos y ajustar parámetros mediante software sin comprar componentes físicos.

Limitaciones físicas frente al hardware real

- Rebote del pulsador (Bounce): En Wokwi la transición de señal es instantánea y limpia. En el hardware real, los contactos metálicos vibran durante milisegundos al presionarse, lo que genera múltiples lecturas falsas si no se implementa debouncing por software o hardware.
- Límites de corriente y potencia: Wokwi no simula daños por sobrecorriente; por ejemplo, un LED conectado sin resistencia limitadora seguirá funcionando en la pantalla. En el hardware real, superar los límites de corriente quema los pines I/O del microcontrolador o los periféricos.
- Tiempos de ejecución: En la simulación, la velocidad depende del rendimiento del procesador de la computadora y del navegador. En el hardware real, los tiempos son estrictamente deterministas, dictados por el reloj de cristal del microcontrolador.
- Comportamiento analógico: Wokwi trabaja con valores matemáticos idealizados. El hardware real está expuesto a ruido electromagnético, caídas de tensión en la fuente y variaciones impredecibles en las lecturas del convertidor analógico-digital (ADC).

7. Conclusiones

- Se pudo reconocer los diferentes tipos de pines dentro de la placa Arduino UNO, así mismo, el comportamiento del código es visualmente similar a pesar de estar dando instrucciones distintas y que a nivel interpretativo puede ser más o menos eficiente.
- Se pudo comparar resultados calculados a mano y ejecutados mediante codificación, mostrando cómo los valores calculados son exactamente iguales a los obtenidos en la práctica simulada.
- Se observó el acceso que se tiene mediante la API de Arduino con el acceso directo a registros, en tiempo de ejecución, ya que mientras el acceso directo da una mayor rapidez y optimización la API de Arduino da más estabilidad entre diferentes modelos siendo portable y facilidad.
- Además, se vio el uso de la plataforma de Wokwi y sus múltiples herramientas de simulación, tanto como sus ventajas y desventajas de su uso.



8. Declaración de IA (Inteligencia Artesanal)

- Los autores declaran haber utilizado IA generativa en el proceso de investigación y redacción. De acuerdo con la taxonomía GAIDeT (2026), se delegaron las siguientes tareas a herramientas de IA generativa bajo supervisión humana total:
 - Corrección, edición y mejora de la redacción del presente documento
 - Estructurar con mayor claridad la explicación teórica de los comportamientos observados en las simulaciones
- La herramienta de IA generativa utilizada fue: ChatGPT GPT-5. La responsabilidad del manuscrito final recae íntegramente en los autores. Las herramientas de IA generativa no figuran como autores y no asumen responsabilidad alguna por los resultados finales.
- Declaración presentada por: Responsabilidad colectiva

9. Anexos

<https://wokwi.com/projects/476992066822540289>

<https://wokwi.com/projects/476993063522443265>